

실시간 운영체제 기반 전자제어기 프로그래밍

Contents

실시간 운영체제 기반 전자제어기
프로그래밍

FreeRTOS 커널 배포판

FreeRTOS 커널 배포판

Introduction

□ 본 장의 구성

- FreeRTOS 커널의 파일과 디렉터리 구조
 - FreeRTOS 디렉터리 구조의 상위 수준 구성
 - 특정 FreeRTOS 프로젝트에 필요한 소스 파일
 - 데모 애플리케이션(Demo Applications)
 - 새로운 FreeRTOS 프로젝트를 만드는 방법

FreeRTOS 배포판 이해

□ FreeRTOS Port

○ FreeRTOS가 지원하는 “컴파일러 + 프로세서 아키텍처”의 각 조합

- FreeRTOS는 약 20종의 서로 다른 컴파일러로 빌드할 수 있으며, 40종이 넘는 프로세서 아키텍처에서 실행할 수 있다.

□ FreeRTOS 빌드

○ 개요

- FreeRTOS는 원래 단일 스레드로 동작하는 Bare-Metal 애플리케이션에 멀티태스킹 기능을 제공하는 라이브러리이다.
- FreeRTOS는 여러 개의 C 소스 파일로 제공되며, 일부 소스 파일은 모든 Port에서 공통으로 사용하고, 일부는 특정 Port에만 사용된다.
- 필요한 소스 파일을 애플리케이션 프로젝트에 포함하여 함께 빌드하면 FreeRTOS API를 사용할 수 있다.

FreeRTOS 배포판 이해

□ FreeRTOSConfig.h

○ 기능

- FreeRTOSConfig.h는 FreeRTOS 커널의 동작을 애플리케이션에 맞게 설정하는 구성 헤더 파일이다.
- 이 파일에 정의된 상수들을 이용해 커널 기능을 선택하고 조정합니다.

○ 중요!!

- 소스 파일에서 FreeRTOSConfig.h를 직접 include하지 않으며, 대신 FreeRTOS.h를 include 하면 FreeRTOS.h가 적절한 시점에 FreeRTOSConfig.h를 포함하게 된다.
- 예시
 - configUSE_PREEMPTION 상수는 FreeRTOS가 협력형(Co-operative) 스케줄링을 사용할지, 선점형(Pre-emptive) 스케줄링을 사용할지를 설정한다.
 - FreeRTOSConfig.h는 특정 애플리케이션에 맞춘 설정 파일이므로 FreeRTOS 커널 소스 디렉터리가 아니라 애플리케이션 프로젝트의 디렉터리에 두는 것이 적절하다.
 - 메인 FreeRTOS 배포판에는 각 FreeRTOS Port별 데모 애플리케이션과 FreeRTOSConfig.h가 포함되어 있다. 처음부터 새로 작성하기보다는, 사용하려는 Port의 데모 프로젝트에 포함된 FreeRTOSConfig.h를 복사한 후 필요한 설정을 수정하는 방법을 권장한다.
 - FreeRTOSConfig.h에 모든 설정 상수를 반드시 작성할 필요는 없으며, 생략된 많은 설정 항목은 기본값(default value)을 사용합니다.

실시간 운영체제

□ 공식 배포판

○ Git URL을 통한 다운로드 방법

- `git clone https://github.com/FreeRTOS/FreeRTOS.git --recurse-submodules`
- `git clone git@github.com:FreeRTOS/FreeRTOS.git --recurse-submodules`

○ 메인 배포판의 상위 디렉터리 구조

FreeRTOS

- └─ Source
- └─ Demo

FreeRTOS 커널 소스 파일

Port별로 미리 구성된 FreeRTOS 커널 데모 프로젝트

FreeRTOS-Plus

- └─ Source
- └─ Demo

기타 FreeRTOS 및 생태계 라이브러리 소스 코드

기타 FreeRTOS 및 생태계 라이브러리 데모 프로젝트

- 배포판에는 FreeRTOS 커널 소스 파일이 한 벌만 들어 있으며, 모든 데모 프로젝트는 FreeRTOS/Source 디렉터리에서 커널 소스 파일을 찾도록 구성되어 있다. 따라서 이 디렉터리 구조를 임의로 변경하면 데모 프로젝트가 빌드되지 않을 수 있다.

실시간 운영체제

□ 모든 Port에 공통인 FreeRTOS 소스 파일

○ 파일명 및 역할

- tasks.c와 list.c는 FreeRTOS 커널의 핵심 기능을 구현하므로 항상 필요하다. 이 파일들은 FreeRTOS/Source 디렉터리에 있으며, 같은 디렉터리에는 다음과 같은 선택적으로 사용할 수 있는 소스 파일도 있다.

파일명	역할	사용 여부
tasks.c	FreeRTOS 커널 핵심 Task 관리 기능	항상 필요
list.c	커널 내부 리스트 자료구조 및 핵심 기능	항상 필요
queue.c	Queue와 Semaphore 서비스 제공	거의 항상 필요
timers.c	Software Timer 기능 제공	Software Timer 사용 시
event_groups.c	Event Group 기능 제공	Event Group 사용 시
stream_buffer.c	Stream Buffer와 Message Buffer 기능 제공	해당 Buffer 사용 시
croutine.c	Co-routine 기능 구현	선택 사항; 현재 유지보수 중단

- Co-routine은 매우 작은 마이크로컨트롤러를 위해 설계된 기능으로 현재는 거의 사용되지 않는다. 더 이상 유지보수되지 않으며 새로운 설계에서는 사용을 권장하지 않는다.

실시간 운영체제

□ 모든 Port에 공통인 FreeRTOS 소스 파일 (계속)

○ 파일명 및 역할 (계속)

FreeRTOS	
└─ Source	
├─ tasks.c	항상 필요
├─ list.c	항상 필요
├─ queue.c	거의 항상 필요
├─ timers.c	선택 사항
├─ event_groups.c	선택 사항
├─ stream_buffer.c	선택 사항
└─ croutine.c	선택 사항 / 유지보수 중단

- ZIP 배포판에 사용되는 파일 이름은 다른 프로젝트에서 이미 사용하는 이름과 충돌할 수 있다. 필요하다면 사용자가 파일 이름을 변경할 수 있지만, FreeRTOS 공식 배포판 자체의 파일 이름은 기존 프로젝트와 FreeRTOS 인식 개발 도구의 호환성을 위해 변경할 수 없다.

실시간 운영체제

□ Port별 FreeRTOS 소스 파일

○ 구성

- FreeRTOS/Source/portable 디렉터리에는 특정 FreeRTOS Port에 필요한 소스 파일이 들어 있다. 이 디렉터리는 먼저 컴파일러별로, 그다음 프로세서 아키텍처별로 구성된다.
- 예를 들어 compiler라는 컴파일러와 architecture라는 프로세서 아키텍처를 사용하는 경우, 공통 FreeRTOS 소스 파일과 함께 다음 디렉터리의 소스 파일을 빌드해야 한다.

FreeRTOS/Source/portable/[compiler]/[architecture]

- FreeRTOS에서는 Heap 메모리 할당 기능도 Portable Layer의 일부로 취급한다. configSUPPORT_DYNAMIC_ALLOCATION을 0으로 설정했다면 프로젝트에 Heap 메모리 할당 구현을 포함하지 않는다.
- 동적 메모리 할당을 사용하는 경우에는 FreeRTOS/Source/portable/MemMang 디렉터리에 제공되는 Heap 구현 중 하나를 프로젝트에 포함하거나, 애플리케이션에서 자체 구현을 제공해야 한다.

FreeRTOS가 제공하는 예제 Heap 메모리 할당 구현은 한 프로젝트에 둘 이상 포함하지 않는다.

실시간 운영체제

□ Port별 FreeRTOS 소스 파일 (계속)

○ 구성 (계속)

```
FreeRTOS
├── Source
│   ├── portable
│   │   ├── MemMang
│   │   │   └── 대체 Heap 메모리 할당 구현
│   │   ├── [compiler 1]
│   │   │   ├── [architecture 1]
│   │   │   ├── [architecture 2]
│   │   │   └── [architecture 3]
│   │   └── [compiler 2]
│   │       ├── [architecture 1]
│   │       └── [architecture 2]
│   └── ...
```

실시간 운영체제

□ Include 경로

○ FreeRTOS 빌드를 위한 컴파일러 Include Path 지정

1. FreeRTOS 커널의 핵심 헤더 파일 경로: FreeRTOS/Source/include
2. 사용하는 Port의 소스 파일 경로: FreeRTOS/Source/portable/[compiler]/[architecture]
3. 올바른 FreeRTOSConfig.h가 위치한 디렉터리

□ 헤더 파일

○ 헤더 파일 include 방법

- FreeRTOS API를 사용하는 소스 파일은 먼저 FreeRTOS.h를 include하고, 그 다음 사용하려는 API 함수의 프로토타입이 선언된 헤더 파일을 include해야 한다.

```
#include "FreeRTOS.h"
#include "task.h"      // Task API 사용 시
#include "queue.h"     // Queue API 사용 시
#include "semphr.h"    // Semaphore / Mutex API 사용 시
```

- API 종류에 따라 task.h, queue.h, semphr.h, timers.h, event_groups.h, stream_buffer.h, message_buffer.h 또는 croutine.h를 사용한다. FreeRTOSConfig.h는 FreeRTOS.h가 자동으로 포함하므로 직접 include하지 않는다.

데모 애플리케이션

□ 개요

○ 각 FreeRTOS Port별 데모 애플리케이션 제공

- 작성 당시에는 컴파일 오류나 경고 없이 바로 빌드되도록 구성되어 있다.
- Cross platform support

FreeRTOS는 Windows, Linux, macOS 환경과 다양한 Embedded/일반 Toolchain에서 개발되고 테스트된다. 다만 Toolchain 버전 차이나 테스트 누락으로 인해 빌드 오류가 발생할 수 있다.

○ 제공 목적

- FreeRTOS API 사용 방법 시연
- FreeRTOS 커널 구현에 대한 Stress Test 수행
- 실제 애플리케이션을 개발할 때 출발점으로 활용
- 최소한의 설정과 사전 지식만으로 바로 실험할 수 있는 환경 제공
- 필요한 파일과 컴파일러 옵션이 올바르게 설정된 동작 가능한 프로젝트 예시 제공

데모 애플리케이션

□ 개요 (계속)

○ 구성

- 각 Demo Project는 FreeRTOS/Demo 아래의 고유한 하위 디렉터리에 위치하며, 디렉터리 이름은 해당 데모가 어떤 Port를 대상으로 하는지 나타낸다.
- 각 데모의 안내 페이지에서는 프로젝트 파일의 위치, 대상 하드웨어 또는 에뮬레이터, 하드웨어 설정 방법, 빌드 방법, 예상 동작 등을 확인할 수 있다.
- 모든 데모 프로젝트는 FreeRTOS/Demo/Common/Minimal에 구현된 “common demo tasks” 중 일부를 생성한다. 이 Task들은 실제 응용 기능을 제공하기 위한 것이 아니라 FreeRTOS API 사용법을 보여주고 커널 Port를 테스트하기 위해 존재한다.
- 많은 데모 프로젝트는 간단한 Blinky 형태의 시작 프로젝트로도 구성할 수 있으며, 일반적으로 두 개의 RTOS Task와 하나의 Queue를 생성한다.
- 모든 데모 프로젝트에는 main() 함수가 들어 있는 main.c 파일이 있으며, main()은 FreeRTOS 커널을 시작하기 전에 Demo Application의 Task들을 생성한다.

FreeRTOS

└ Demo

├ [Demo x] demo x 프로젝트 파일

├ [Demo y] demo y 프로젝트 파일

├ [Demo z] demo z 프로젝트 파일

└ Common 모든 데모에서 공통으로 빌드하는 파일

FreeRTOS 프로젝트 만들기

□ 제공된 Demo Project 수정하기

○ 개요

- 각 FreeRTOS Port에는 미리 구성된 Demo Application이 있으므로, 새로운 프로젝트는 기존 Demo Project를 수정하여 시작하는 것이 권장된다. 이렇게 하면 필요한 소스 파일, Interrupt Handler, Compiler Option을 올바르게 설정할 가능성이 높아진다.

○ 기존 Demo Project에서 새 애플리케이션을 만드는 절차

이 과정을 마치면 FreeRTOS에 필요한 소스 파일과 설정은 갖추고 있지만 아직 애플리케이션 기능은 정의하지 않은 깨끗한 프로젝트가 만들어진다.

1. 제공된 Demo Project를 열고 정상적으로 빌드되고 실행되는지 확인한다.
2. Demo/Common 디렉터리에 있는 Demo Task 구현 소스 파일을 제거한다.
3. main()에서 prvSetupHardware()와 vTaskStartScheduler()를 제외한 함수 호출을 제거한다.
4. 프로젝트가 여전히 정상적으로 빌드 되는지 확인한다.

○ 새 main() 함수의 기본 템플릿

```
int main( void ) {
    prvSetupHardware(); /* 필요한 하드웨어 초기화 */

    /* --- APPLICATION TASKS CAN BE CREATED HERE --- */

    vTaskStartScheduler(); /* 생성한 Task 실행 시작 */

    for(;; ); /* Heap 부족으로 Scheduler를 시작하지 못한 경우에만 도달 */

    return 0;
}
```

FreeRTOS 프로젝트 만들기

□ 처음부터 새 프로젝트 만들기

○ 개요

- 가능하면 기존 Demo Project를 기반으로 새 프로젝트를 만드는 것이 권장된다. 부득이하게 처음부터 프로젝트를 구성해야 한다면 다음 절차를 따른다.
- 기존 Demo Project에서 새 애플리케이션을 만드는 절차
 1. 선택한 Toolchain에서 아직 FreeRTOS 소스 파일을 포함하지 않은 새 프로젝트를 만든다.
 2. 새 프로젝트가 정상적으로 빌드되고 Target Hardware로 다운로드되며 실행되는지 확인한다.
 3. 기본 프로젝트가 정상 동작한다는 것을 확인한 후 FreeRTOS에 필요한 소스 파일을 추가한다.
 4. 사용 Port의 Demo Project에 포함된 FreeRTOSConfig.h를 새 프로젝트 디렉터리로 복사한다.
 5. FreeRTOS/Source/include, FreeRTOS/Source/portable/[compiler]/[architecture], FreeRTOSConfig.h가 있는 디렉터리를 Include Path에 추가한다.
 6. 관련 Demo Project의 Compiler Setting을 복사한다.
 7. 필요한 FreeRTOS Interrupt Handler를 설치한다. 해당 Port의 안내 페이지와 Demo Project를 참고한다.

FreeRTOS 프로젝트 만들기

□ 처음부터 새 프로젝트 만들기 (계속)

- 프로젝트에 포함해야 하는 FreeRTOS 소스 파일과 위치

파일명	저장 위치
queue.c	FreeRTOS/Source
list.c	FreeRTOS/Source
timers.c	FreeRTOS/Source
event_groups.c	FreeRTOS/Source
stream_buffer.c	FreeRTOS/Source
Port별 모든 C/Assembler 파일	FreeRTOS/Source/portable/[compiler]/[architecture]
heap_n.c (n = 1, 2, 3, 4, 5)	FreeRTOS/Source/portable/MemMang

➤ Heap 메모리 참고

configSUPPORT_DYNAMIC_ALLOCATION이 0이면 Heap 메모리 할당 구현을 프로젝트에 포함하지 않는다. 동적 할당을 사용한다면 heap_n.c 중 하나 또는 사용자가 직접 만든 Heap 구현을 포함해야 한다.

데이터 형식과 코딩 스타일 가이드

□ 데이터 형식

각 FreeRTOS Port에는 고유한 portmacro.h 헤더 파일이 있으며, 이 파일에는 Port별 데이터 형식 정의가 포함된다. 대표적으로 TickType_t와 BaseType_t가 있다.

○ TickType_t

- FreeRTOS는 Tick Interrupt라고 하는 주기적 인터럽트를 사용한다. FreeRTOS 애플리케이션이 시작된 이후 발생한 Tick Interrupt의 횟수를 Tick Count라고 하며, 시간을 측정하는 기준으로 사용한다.
- 두 Tick Interrupt 사이의 시간을 Tick Period라고 하며, FreeRTOS의 시간 값은 일반적으로 Tick Period의 배수로 표현된다. TickType_t는 Tick Count를 저장하고 시간 값을 지정할 때 사용하는 데이터 형식이다.
- TickType_t는 FreeRTOSConfig.h의 configTICK_TYPE_WIDTH_IN_BITS 설정에 따라 unsigned 16-bit, 32-bit 또는 64-bit 형식이 될 수 있다. 이 설정은 아키텍처에 따라 달라지며 각 FreeRTOS Port가 설정값의 유효성을 검사한다.
- 8-bit 또는 16-bit 아키텍처에서는 16-bit TickType_t가 효율을 높일 수 있지만, FreeRTOS API에서 지정할 수 있는 최대 Block Time이 크게 제한된다. 32-bit 또는 64-bit 아키텍처에서는 16-bit TickType_t를 사용할 이유가 없다.
- 기존의 configUSE_16_BIT_TICKS 설정은 32-bit를 초과하는 Tick Count를 지원하기 위해 configTICK_TYPE_WIDTH_IN_BITS로 대체되었다. 새로운 설계에서는 configTICK_TYPE_WIDTH_IN_BITS 사용을 권장한다.

데이터 형식과 코딩 스타일 가이드

□ 데이터 형식 (계속)

○ TickType_t (계속)

설정값	8-bit 아키텍처	16-bit 아키텍처	32-bit 아키텍처	64-bit 아키텍처
TICK_TYPE_WIDTH_16_BITS	uint16_t	uint16_t	uint16_t	N/A
TICK_TYPE_WIDTH_32_BITS	uint32_t	uint32_t	uint32_t	N/A
TICK_TYPE_WIDTH_64_BITS	N/A	N/A	uint64_t	uint64_t

○ BaseType_t

- BaseType_t는 해당 아키텍처에서 가장 효율적인 데이터 형식으로 정의된다. 일반적으로 64-bit 아키텍처에서는 64-bit, 32-bit 아키텍처에서는 32-bit, 16-bit 아키텍처에서는 16-bit, 8-bit 아키텍처에서는 8-bit 형식을 사용한다.
- BaseType_t는 반환값의 종류가 많지 않은 함수의 Return Type이나 pdTRUE/pdFALSE와 같은 Boolean 값에 주로 사용된다.

데이터 형식과 코딩 스타일 가이드

□ 변수 이름

FreeRTOS에서는 변수 이름 앞에 데이터 형식을 나타내는 접두사(prefix)를 붙인다.

접두사	의미/예시
c	char
s	int16_t (short)
l	int32_t (long)
x	BaseType_t 또는 구조체, Task Handle, Queue Handle 등 비표준 형식
u	unsigned 형식에 추가
p	pointer 형식에 추가

- 예를 들어 uint8_t 변수에는 uc 접두사를 사용하고, char* 형식의 포인터 변수에는 pc 접두사를 사용한다.

```
uint8_t ucValue;  
char    *pcString;
```

데이터 형식과 코딩 스타일 가이드

□ 함수 이름

함수 이름의 접두사는 “반환 형식”과 “함수가 정의된 소스 파일”을 함께 나타낸다.

예	의미
vTaskPrioritySet()	void를 반환하며 tasks.c에 정의
xQueueReceive()	BaseType_t를 반환하며 queue.c에 정의
pvTimerGetTimerID()	void*를 반환하며 timers.c에 정의
prv...	파일 내부에서만 사용하는 Private 함수 접두사

□ 포매팅

일부 Demo Application에서는 Tab 한 개를 Space 네 칸으로 설정하여 사용한다. 현재 FreeRTOS Kernel 소스 코드는 더 이상 Tab 문자를 사용하지 않는다.

데이터 형식과 코딩 스타일 가이드

□ 매크로 이름

대부분의 FreeRTOS 매크로는 대문자로 작성하며, 매크로가 어느 파일에서 정의되었는지를 나타내는 소문자 접두사를 앞에 붙인다.

접두사	정의 위치	예
port	portable.h 또는 portmacro.h	portMAX_DELAY
task	task.h	taskENTER_CRITICAL()
pd	projdefs.h	pdTRUE
config	FreeRTOSConfig.h	configUSE_PREEMPTION
err	projdefs.h	errQUEUE_FULL

Semaphore API는 거의 대부분 매크로로 구현되어 있지만, 매크로 이름 규칙이 아니라 함수 이름 규칙을 따른다.

FreeRTOS 소스 코드 전반에서 자주 사용하는 공통 매크로는 다음과 같다.

Macro	Value
pdTRUE	1
pdFALSE	0
pdPASS	1
pdFAIL	0

데이터 형식과 코딩 스타일 가이드

□ 과도한 Type Casting을 사용하는 이유

FreeRTOS 소스 코드는 매우 다양한 컴파일러에서 빌드된다. 컴파일러마다 Warning을 발생시키는 조건과 Casting을 요구하는 방식이 서로 다르다.

그 결과 FreeRTOS 소스 코드에는 일반적인 코드보다 많은 Type Casting이 포함되어 있다. 이는 여러 컴파일러에서 일관되게 빌드하고 불필요한 Warning을 줄이기 위한 호환성 목적의 선택이다.